

CoGS-SLAM: Correspondence-Guided Gaussian Splatting SLAM with One-Shot Dense Initialization

Wei-Tse Cheng
National Yang Ming Chiao Tung University
andy5552555.ii13@nycu.edu.tw

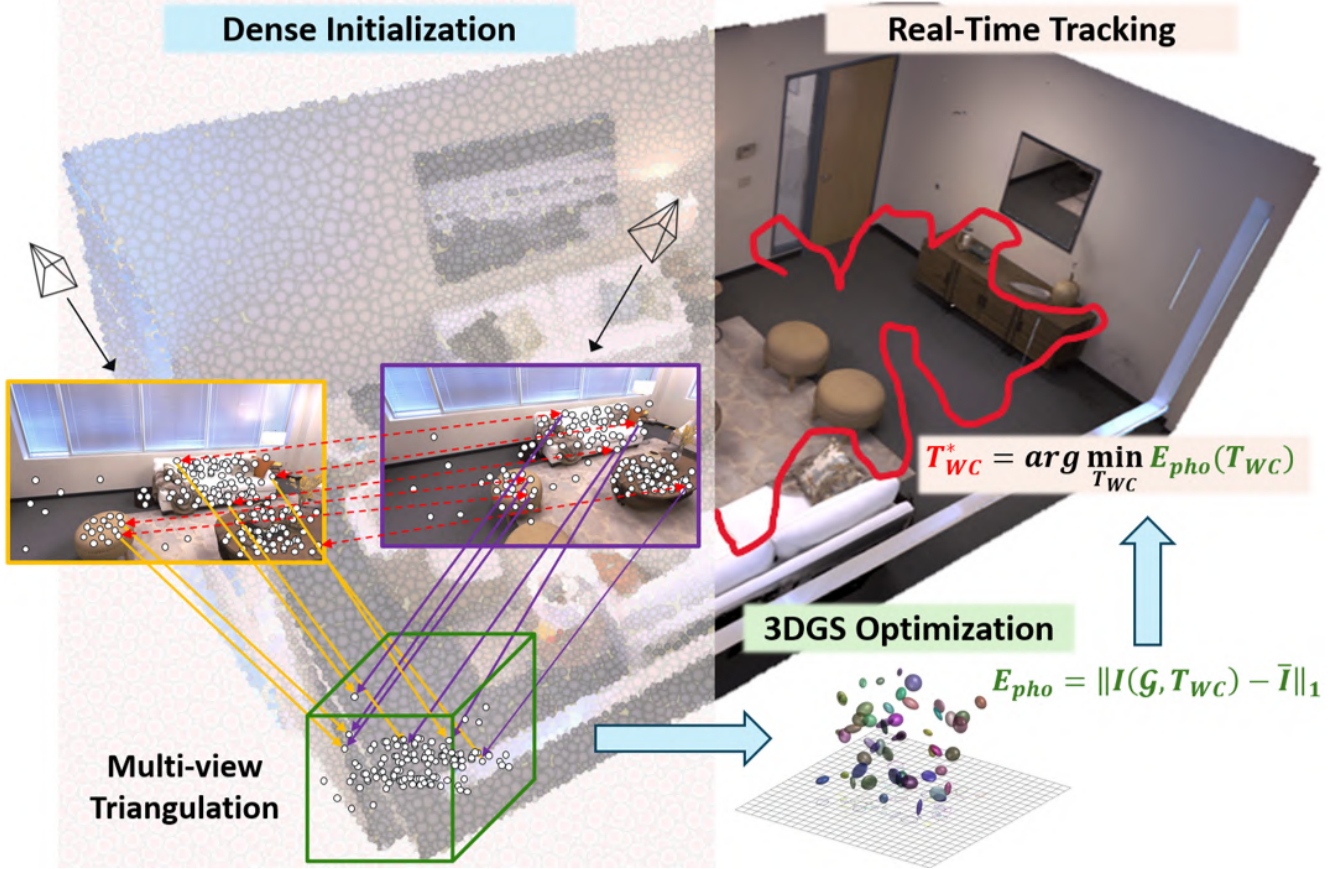


Figure 1. Overview of the proposed CoGS-SLAM pipeline. The system integrates dense feature matching and multi-view triangulation for one-shot Gaussian initialization, followed by differentiable 3DGS optimization and real-time tracking.

Abstract

We present CoGS-SLAM, a Gaussian-splatting SLAM system that substitutes the residual-based densification module in GS-SLAM with a training-free correspondence-guided Gaussian initialization scheme. Rather than incrementally inserting Gaussians when image residuals expose missing regions, CoGS-SLAM establishes dense multi-view correspondences from DINOv3 features, filters them with a confidence-aware inlier classifier, and triangulates them in a single step to produce a well-covered and geometry-aware

Gaussian prior before optimization. This initialization improves the stability of early-stage mapping and reduces convergence time by about 20%, while delivering stronger rendering quality in textured and cluttered environments and preserving compatibility with GS-SLAM frameworks. Experiments on TUM and Replica show that CoGS-SLAM attains competitive or better localization and reconstruction performance than Gaussian- and point-based SLAM methods, while maintaining real-time mapping throughput of up to 925 FPS.

1. Introduction

Recent advances in 3D Gaussian Splatting (3DGS) have enabled high-quality view synthesis and real-time mapping. However, most pipelines still rely on residual-driven densification, where Gaussians are iteratively spawned and merged as errors are detected. This causes non-stationary objectives, unstable convergence, and sensitivity to texture-rich or cluttered regions due to delayed coverage and uneven geometry.

We take a different approach by initializing from a complete and well-distributed Gaussian set rather than growing it incrementally. Using Dense Feature Matching (DFM), we obtain confidence-weighted correspondences within a short keyframe window, triangulate them into structure-aware matches, and instantiate the Gaussian set before optimization begins. Subsequent updates refine means, covariances, opacities, and colors while keeping topology fixed, resulting in stable and stationary optimization with strong spatial support even in high-frequency regions.

Integrated into monocular SLAM, this single-step seeding shortens time to usable maps, stabilizes early pose and shape estimation, and removes the need for additional networks or losses. It directly replaces densification with a brief feature-matching pass at keyframes while leaving the rest of the pipeline unchanged. An overview of the CoGS-SLAM framework, including the initialization pipeline, is shown in Figure 1. Our contributions are summarized below.

- **Single-Step Dense Initialization.** A one-shot triangulation replaces residual-driven densification within the standard GS-SLAM pipeline, enabling stationary optimization and 20% faster convergence.
- **Improved Localization Accuracy.** Confidence-weighted correspondences stabilize early pose estimation, reducing drift by over 30%.
- **Lightweight and Efficient Mapping.** Spatially balanced Gaussians lower computation and memory, achieving 20% higher rendering throughput in real time.
- **High-Fidelity Reconstruction.** Dense seeding enhances early coverage and geometric consistency, yielding about 20% better reconstruction accuracy and completeness.

2. Related Work

3D Gaussian Splatting and Densification. 3D Gaussian splatting (3DGS) enables real-time view synthesis with anisotropic splats and a visibility-aware renderer, yet most systems rely on residual-driven densification [8]. We instead seed a fixed topology from dense multi-view correspondences and then refine only splat parameters.

Gaussian Splats for SLAM. SLAM systems mapping with Gaussians include GS-SLAM, MonoGS, SplatAM, and Gauss-SLAM [7, 11, 20]. They rely on densification, caus-

ing early non-stationarity. Our training-free dense seed removes this stage and drops into MonoGS with minimal modifications.

Differentiable Rendering and Real-Time SLAM. PhotoSLAM, GLORIE-SLAM, and Point-SLAM couple differentiable rendering with pose optimization for fast updates via analytic/lightweight gradients [4, 12, 23]. We retain this in a Gaussian renderer and use a stationary initialization so early steps do not change topology.

Feature Matching and Dense Correspondence. SuperPoint/SuperGlue remain strong baselines [1, 13]. Detector-free transformers (LoFTR, LightGlue) extend coverage in low-texture regions, and dense matchers (DKM) provide broad two-view coverage with confidence for geometry [2, 9, 17]. We aggregate confidence-weighted dense correspondences over a short keyframe window into an explicit Gaussian seed.

Initialization and Training-Free Priors. SfM and multi-view stereo stabilize early optimization via geometric priors and learned depth [14, 22]. In Gaussian splatting, schedules and regularizers typically retain densification. Our correspondence-to-Gaussian initialization follows training-free priors and yields a stationary objective from the start.

3. Method

3.1. Gaussian Splatting Representation

We map the scene with a set of anisotropic Gaussians $\mathcal{G} = \{G_i\}$. Each G_i carries optical properties, a color vector c_i and an opacity $\alpha_i \in [0, 1]$, and geometric properties, a mean $\mu_i^W \in \mathbb{R}^3$ and a symmetric positive definite covariance $\Sigma_i^W \in \mathbb{R}^{3 \times 3}$ expressed in world coordinates. For brevity we describe color as a single vector and later allow spherical harmonics for view dependence.

Let $T_{WC} = [R \mid t]$ be the world-to-camera pose of the current view and let $\pi(\cdot)$ be the calibrated perspective projection. A 3D Gaussian $N(\mu_i^W, \Sigma_i^W)$ induces a 2D Gaussian on the image plane through first-order linearization of the projection around μ_i^W . The projected mean and covariance are

$$\mu_i^I = \pi(T_{WC} \mu_i^W), \quad \Sigma_i^I = J_i R \Sigma_i^W R^\top J_i^\top, \quad (1)$$

where $J_i = \frac{\partial \pi(RX+t)}{\partial X} \big|_{X=\mu_i^W}$ is the Jacobian of the projection at μ_i^W . This relates the ellipsoid in 3D to an ellipsoid on the sensor.

Rendering is performed by rasterizing Gaussians instead of ray marching. For pixel p we collect the front-to-back ordered set $N(p)$ of screen-space Gaussians whose 2D footprints overlap p . The pixel color is obtained by α -compositing

$$C_p = \sum_{i \in N(p)} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (2)$$

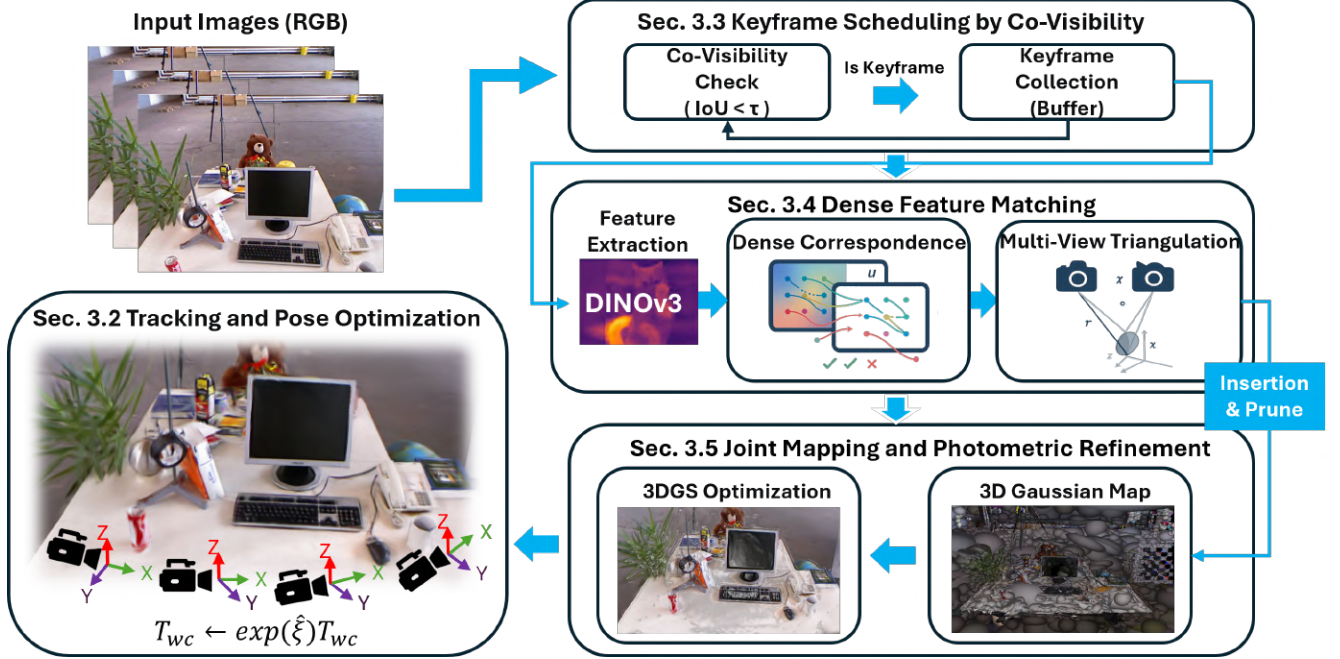


Figure 2. Detailed CoGS-SLAM pipeline. Each keyframe triggers dense multi-view triangulation that yields a one-shot Gaussian initialization, subsequently refined through joint tracking and mapping within a differentiable 3DGS renderer using analytic SE(3) Jacobians.

where $\alpha_i \in [0, 1]$ denotes the *screen-space* opacity at pixel p , obtained by modulating the primitive opacity with the value of the 2D Gaussian density at p using the parameters (μ_i^I, Σ_i^I) (the dependence on p is omitted for brevity). Empty space does not contribute because the renderer iterates over primitives that actually cover the pixel.

Equations (1) and (2) are differentiable in color, opacity, mean, covariance, and pose. Gradients flow through the splat weights and the projection Jacobian, which allows first-order optimizers to refine both optical and geometric parameters until the rendered image matches the observation with high fidelity. In subsequent sections we will initialize $\{\mu_i^W, \Sigma_i^W, \alpha_i, c_i\}$ densely in a single-step and then refine them under this differentiable renderer.

3.2. Tracking and Camera Pose Optimization

3.2.1. Objective and Per-Frame Update

For each incoming frame, we estimate the camera pose by minimizing an image-domain objective under the 3DGS renderer in Figure 2. Let $I(\mathcal{G}, T_{WC}) = \mathcal{S}(\mathcal{G}, T_{WC})$ be the rendered image and $\bar{I}$ the observation. The photometric residual is

$$E_{\text{pho}} = \|I(\mathcal{G}, T_{WC}) - \bar{I}\|_1, \quad (3)$$

augmented with an affine brightness model to absorb exposure changes, i.e., we jointly estimate gain and bias and substitute $gI(\cdot) + b$ into Eq. (3). We optimize $E = E_{\text{pho}}$.

Pixels with low screen-space opacity or low image gradient are downweighted to reduce the influence of textureless regions. In practice we perform tens of gradient steps per frame to reach a stable update.

3.2.2. Alpha Compositing for Color

Rendering is carried out by rasterizing Gaussians in screen space and compositing them front to back. For a pixel p , let $N(p)$ be the set of overlapping Gaussians sorted from near to far. The color follows the standard α -compositing in Eq. (2), which naturally handles occlusion via transmittance $\prod_{j < i} (1 - \alpha_j)$. No depth map is produced or used in our tracking objective.

3.2.3. Minimal Pose Jacobians on SE(3)

We update the world-to-camera pose by a left-multiplicative twist SE(3),

$$T_{WC} \leftarrow \exp(\hat{\xi}) T_{WC}, \quad (4)$$

where we differentiate the objective with respect to ξ in minimal coordinates. Let μ^W be a 3D Gaussian mean in world coordinates and $\mu^C = R\mu^W + t$ its camera-space position for $T_{WC} = [R | t]$. The 3D point Jacobian with respect to the pose twist is the standard 3×6 form

$$\frac{\partial \mu^C}{\partial \xi} = [I \quad -[\mu^C]_{\times}], \quad (5)$$

where $[\cdot]_{\times}$ is the skew-symmetric matrix. With calibrated projection π , the image-plane mean $\mu^I = \pi(\mu^C)$ has Jaco-

bian

$$\frac{\partial \mu^I}{\partial \xi} = J_\pi(\mu^C) [I - [\mu^C]_\times], \quad (6)$$

where J_π is the 2×3 projection Jacobian evaluated at μ^C . The screen-space covariance Σ^I from Eq. (1) depends on both the projection Jacobian and the rotation, using the chain rule,

$$\frac{\partial \Sigma^I}{\partial \xi} = \frac{\partial \Sigma^I}{\partial J} \frac{\partial J}{\partial \mu^C} \frac{\partial \mu^C}{\partial \xi} + \frac{\partial \Sigma^I}{\partial R} \frac{\partial R}{\partial \xi}, \quad (7)$$

with $\partial R / \partial \xi$ obtained from the Lie algebra relation $\delta R \approx [\delta \omega]_\times R$ for an infinitesimal rotation $\delta \omega$. These analytic Jacobians remove the overhead of generic autodiff and match the degrees of freedom of the pose, which is essential for fast and stable tracking under a tight per-frame budget.

3.2.4. Optimization Solver and Weighting Scheme

We minimize the photometric objective in Eq. (3) using a first-order optimizer with a cosine learning rate schedule, and apply a robust penalty to per-pixel residuals. The per-pixel weight combines exposure correction, edge awareness, and visibility (via screen-space opacity) so that informative regions dominate the update. Because the 3DGS renderer and the pose Jacobians are fully analytic, gradients propagate through Eq. (2) and Eq. (6) without resorting to expensive automatic differentiation.

3.3. Keyframe Scheduling by Co-Visibility

Given the last accepted keyframe I_{k^*} , we measure co-visibility between the current frame I_k and I_{k^*} by the intersection-over-union of visible Gaussians

$$\text{IoU}(I_k, I_{k^*}) = \frac{|V(I_k) \cap V(I_{k^*})|}{|V(I_k) \cup V(I_{k^*})|}, \quad (8)$$

where $V(I)$ collects Gaussians whose screen-space opacity exceeds a small threshold on a sufficient fraction of pixels. A new keyframe is created when $\text{IoU}(I_k, I_{k^*})$ is less than τ and the inter-view parallax is above a bound. Accepted keyframes are stored in a bounded buffer $\mathcal{B}$ that provides neighbours for multi-view initialization.

3.4. Dense Feature Matching

We extract dense visual descriptors using DINOv3 [15], which provide semantically consistent features across views. These descriptors are used to establish multi-view dense correspondences, replacing the residual-driven densification process in GS-SLAM. A confidence-aware inlier classifier is then applied to filter unreliable matches, ensuring stable multi-view geometry. Finally, a one-shot triangulation is performed to initialize a uniformly distributed set of 3D Gaussian seeds.

Dense Correspondence. Let I_r be the current keyframe and let $\mathcal{N}_r \subset \mathcal{B}$ be K neighbours selected by pose proximity and parallax. A dense matcher outputs, for each pair (r, n) with $n \in \mathcal{N}_r$, a displacement field $\mathbf{u}_{r \rightarrow n}(p)$ on I_r and a confidence map $\kappa_{r \rightarrow n}(p) \in [0, 1]$. A correspondence is represented as the pixel pair

$$(p, p + \mathbf{u}_{r \rightarrow n}(p)), \quad (9)$$

and low-confidence matches are filtered by a symmetric epipolar test and spatial blue-noise thinning. We aggregate per-view confidence for each retained reference pixel by

$$\bar{\kappa}(p) = \frac{1}{|\mathcal{N}_r|} \sum_{n \in \mathcal{N}_r} \kappa_{r \rightarrow n}(p). \quad (10)$$

Multi-view Triangulation. For each retained pixel p and neighbour $n \in \mathcal{N}_r$, we solve a two-view linear triangulation. Let $P_r, P_n \in \mathbb{R}^{3 \times 4}$ be the camera projection matrices and $\tilde{x}_r, \tilde{x}_n \in \mathbb{P}^2$ the homogeneous pixel coordinates. We form

$$A = \begin{bmatrix} \tilde{x}_r^x P_r^{3\top} - P_r^{1\top} \\ \tilde{x}_r^y P_r^{3\top} - P_r^{2\top} \\ \tilde{x}_n^x P_n^{3\top} - P_n^{1\top} \\ \tilde{x}_n^y P_n^{3\top} - P_n^{2\top} \end{bmatrix}, \quad \tilde{X} = \arg \min_{\|\tilde{X}\|=1} \|A\tilde{X}\|_2,$$

then $\hat{X} = \tilde{X}_{1:3} / \tilde{X}_4$ (obtained as the right singular vector of A associated with the smallest singular value). Among all neighbours we keep the hypothesis with the lowest reprojection error, breaking ties by larger baseline angle. Candidates with small parallax or large reprojection error are rejected.

Gaussian Parameter Initialization. Each valid triangulation spawns one Gaussian G_i with world mean

$$\mu_i^W = \hat{X}(p). \quad (11)$$

Construct a local orthonormal frame $U_i = [\mathbf{t}_1, \mathbf{t}_2, \mathbf{v}]$, where $\mathbf{v}$ is the surface normal estimated by a plane fit over neighbouring triangulated points, and $\mathbf{t}_1, \mathbf{t}_2$ span the tangent plane. Initialize the covariance as an anisotropic ellipsoid aligned with this frame

$$\Sigma_i^W = U_i \text{diag}(s_\perp^2, s_\perp^2, s_\parallel^2) U_i^\top, \quad (12)$$

where $s_\perp$ is obtained by back-projecting a one-pixel image uncertainty via the projection Jacobian at the reference view, and $s_\parallel$ is set larger to encode depth uncertainty that increases when the baseline angle is small or the triangulation residual is high. The color c_i is the median of bilinearly sampled RGB values across supporting views after applying the exposure parameters estimated by tracking. The initial opacity α_i is a monotone mapping of the aggregated correspondence confidence $\bar{\kappa}(p)$ so that unreliable candidates remain visually weak at insertion. Finally, Gaussians are subsampled to maintain uniform spatial coverage before being inserted into the map.

3.5. Joint Mapping and Photometric Refinement

At each accepted keyframe, we first perform the single-step dense initialization (Sec. 3.4) to generate Gaussian seeds from multi-view correspondences. The surviving seeds are immediately inserted into $\mathcal{G}$ *without* any densification stage. Each newly inserted Gaussian participates in mapping right away.

Insertion, Lightweight Merging, and Pruning. Each Gaussian G_i tracks its observation count m_i , cumulative visibility v_i , and an exponential moving average of its screen-space footprint. To keep memory bounded and remove unstable outliers, we apply a lightweight periodic cleanup and prune splats that violate

$$\begin{aligned} m_i &< m_{\min}, & v_i &< v_{\min}, \\ \text{tr}(\Sigma_i^W) &> \sigma_{\max}^2, & \alpha_i &< \alpha_{\min}. \end{aligned}$$

Neighbouring Gaussians with highly overlapping footprints and similar colors are merged, retaining a visibility-weighted mean of color and covariance to avoid overpopulation.

Sliding-Window Photometric Refinement. Let $\mathcal{W}$ denote a window around the latest keyframe. We jointly refine $\{T_{WC}\}_{I \in \mathcal{W}}$ and $\{c_i, \alpha_i, \mu_i^W, \Sigma_i^W\}$ by minimizing

$$\mathcal{L} = \sum_{I \in \mathcal{W}} \lambda_{\text{pho}} \|g_I \mathcal{S}(\mathcal{G}, T_{WC}) + b_I - \bar{I}\|_1 + \mathcal{R}, \quad (13)$$

where g_I, b_I compensate exposure changes. The regularizer

$$\begin{aligned} \mathcal{R} = & \lambda_{\text{iso}} \sum_i \left\| \Sigma_i^W - \frac{\text{tr}(\Sigma_i^W)}{3} I_3 \right\|_F \\ & + \lambda_{\alpha} \sum_i \psi(\alpha_i) + \lambda_{\mu} \sum_i \|\mu_i^W - \bar{\mu}_i^W\|_2^2 \end{aligned} \quad (14)$$

discourages needle-shaped ellipsoids, avoids degenerate transmittance, and stabilizes early iterations via an EMA anchor $\bar{\mu}_i^W$. We alternate pose-only updates and full map updates with robust per-pixel weights. Gradients propagate through the analytic α -compositing in Eq. (2) and the pose Jacobians in Eq. (6).

3.6. System Schedule and Computational Profile

Each incoming frame is tracked for K_t gradient steps using the photometric objective in Eq. (3). When the co-visibility test Eq. (23) accepts a keyframe, we select K neighbours from $\mathcal{B}$ and execute the single-step dense initialization of Sec. 3.4 (dense correspondence, weighted multi-view triangulation, and parameter initialization) in one pass. The resulting Gaussians are immediately inserted into $\mathcal{G}$, followed by K_m mapping iterations over the current window $\mathcal{W}$ optimizing Eq. (13), and a lightweight cleanup as described in Sec. 3.5. Replacing iterative densification with this single-step initialization reduces the drift of newly added parameters and lowers the number of mapping iterations required to

reach the same photometric fidelity, improving wall-clock throughput without changing the objective or renderer.

4. Experiments

4.1. Experimental Setup

Datasets. We evaluate on TUM RGB-D and Replica. TUM RGB-D is evaluated in both monocular and RGB-D settings. Replica is employed for photometric map evaluation on *room0–2* and *office0–4*, matching the splits used in our tables to ensure comparability of rendering metrics and throughput.

Implementation. Gaussian rasterization and gradients are implemented in CUDA, and the remaining pipeline is in PyTorch. Mixed precision is enabled where beneficial. Tracking runs in real time, while mapping executes asynchronously within a bounded local window. Non-standard hyperparameters (learning rate schedule, window sizes, keyframe and culling thresholds) are provided in the supplementary.

Evaluation Metrics. Tracking accuracy uses RMSE of Absolute Trajectory Error (ATE) on keyframes. Photometric quality adopts PSNR [3], SSIM [19], and LPIPS [24]. Reconstruction quality reports *Acc.* [cm]↓, *Comp.* [cm]↓, and *Comp.Ratio* (%)↑. Unless specified, we uniformly sample 50K surface points, set $\tau = 5$ cm, and average per scene. Photometric metrics are computed on every fifth frame excluding keyframes. Reconstruction metrics use the same sampling protocol. Each experiment is repeated three times, and the mean results are reported.

Baseline Methods. We compare with iMAP [16], NICE-SLAM [26], Vox-Fusion [21], ESLAM [6], Point-SLAM [12], Co-SLAM [18], SplaTAM [7], Gauss-SLAM [20], and MonoGS [11]. We also include SNI-SLAM [25] for reconstruction and Photo-SLAM [4], GLORIE-SLAM [23], RK-SLAM [10] for rendering. RGB-D-only methods run in RGB-D, with monocular results reported only when supported. Hyperparameters follow official documented defaults on the same splits.

4.2. Training and Convergence Analysis

The system optimizes scene specific Gaussian parameters and camera poses using the same optimizer, schedule, and window size as MonoGS, with densification removed. Each frame is tracked for K_t steps. Each accepted keyframe triggers the single-step dense initialization followed by K_m mapping steps with exposure compensation. Wall clock time is measured on identical hardware and stopping criteria. Results are summarized in Table 1. On TUM RGB-D the average training time decreases from 14.8 to 12 minutes while maintaining localization and rendering quality.

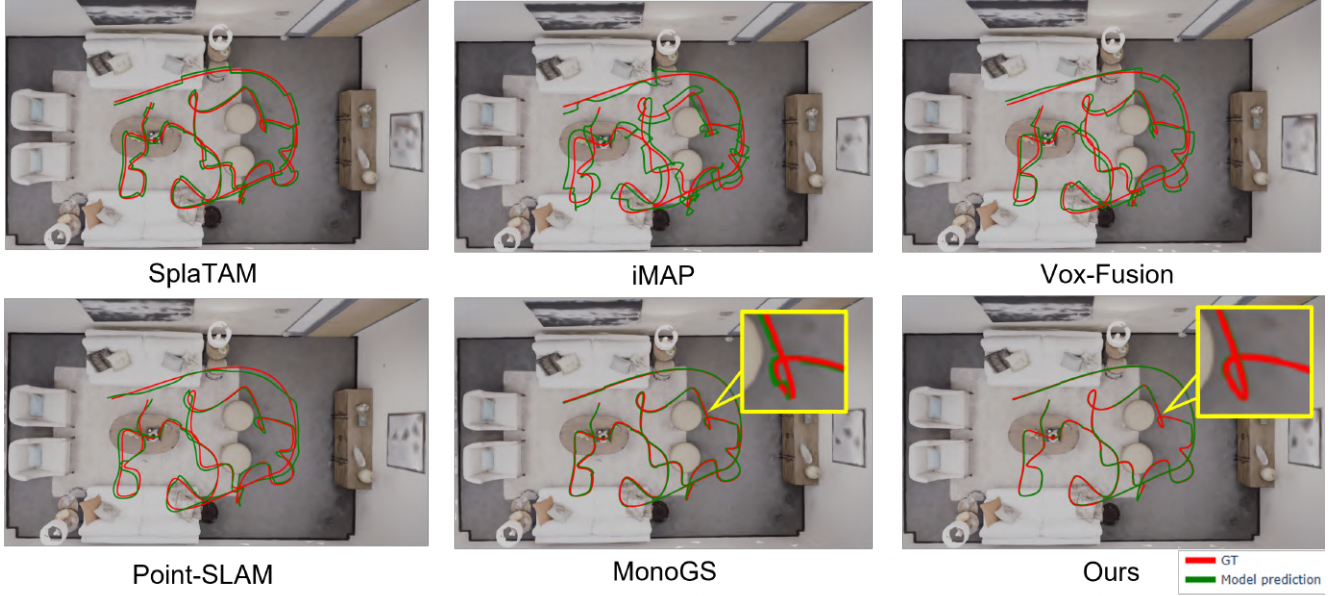


Figure 3. Trajectory comparison on a living-room scene. The red line indicates the ground-truth path and the green line shows the estimated trajectory. Our method aligns more closely with the ground-truth and exhibits fewer large drifts than previous systems.

Table 1. Optimization time (min) on TUM RGB-D sequences *fr1/desk*, *fr2/xyz*, and *fr3/office*.

Method	<i>fr1/desk</i>	<i>fr2/xyz</i>	<i>fr3/office</i>	Avg.
MonoGS [11]	6.4	20.6	17.5	14.8
Ours (RGB)	4.9	16.1	15.0	12.0

4.3. Localization Accuracy

We evaluate trajectory accuracy on Replica and TUM (Tables 2, 3). On Replica, the average ATE is **0.61 cm** across *r0-r2*, *o0-o4*, outperforming iMAP (2.58), NICE-SLAM (1.07), Vox-Fusion (3.09), and ESLAM (0.90 cm), while remaining competitive with Point-SLAM and MonoGS. On TUM RGB-D, the average ATE is **1.02 cm** on *fr1/desk*, *fr2/xyz*, and *fr3/office*, achieving better results than MonoGS and surpassing the same baselines. Figure 3 shows a trajectory comparison on a living-room scene from the Replica dataset, where the red line indicates the ground-truth (GT) path and the green line represents the estimated trajectory. Among existing methods, SplaTAM, iMAP, Vox-Fusion, and Point-SLAM exhibit large localization drift, with evident deviations from the GT path. MonoGS and our method perform significantly better. In this visualization, the red line is drawn above the green line, so greater overlap, where the red line covers the green one, intuitively reflects smaller localization error. Our method achieves a higher overlap ratio, indicating closer adherence to the GT trajectory and better pose consistency.

The zoom-in comparison further shows smoother alignment and reduced drift compared with MonoGS, demonstrating stronger robustness in long-term tracking and loop-closure maintenance.

Table 2. Camera tracking results on the Replica dataset under the RGB-D setting. Reported values denote RMSE of ATE across *room0-2* and *office0-4*.

Method	room0	room1	room2	office0	office1	office2	office3	office4	Avg.
iMAP [16]	3.12	2.54	2.31	1.69	1.03	3.99	4.05	1.93	2.58
NICE-SLAM [26]	0.97	1.31	1.07	0.88	1.00	1.06	1.10	1.13	1.07
Vox-Fusion [21]	1.37	4.70	1.47	8.48	2.04	2.58	1.11	2.94	3.09
ESLAM [6]	0.71	0.70	0.52	0.57	0.55	0.58	0.72	0.63	0.63
Point-SLAM [12]	0.61	0.41	0.37	0.38	0.48	0.54	0.69	0.72	0.53
MonoGS [11]	0.62	0.62	0.77	0.44	0.52	0.23	0.62	2.25	0.76
Ours (RGB)	0.45	0.51	0.53	0.52	0.78	1.03	0.45	0.63	0.61

Table 3. Camera tracking results on the TUM RGB-D dataset. Values denote RMSE of ATE over *fr1/desk*, *fr2/xyz*, and *fr3/office*.

Method	<i>fr1/desk</i>	<i>fr2/xyz</i>	<i>fr3/office</i>	Avg.
iMAP [16]	4.90	2.00	5.80	4.23
NICE-SLAM [26]	4.26	6.19	3.87	4.77
DI-Fusion [5]	4.40	2.00	5.80	4.07
Vox-Fusion [21]	3.52	1.49	26.01	10.34
ESLAM [6]	2.47	1.11	2.42	2.00
Co-SLAM [18]	2.40	1.70	2.40	2.17
Point-SLAM [12]	4.34	1.31	3.48	3.04
MonoGS [11]	1.50	1.44	1.49	1.47
Ours (RGB)	1.02	0.98	1.05	1.02



Figure 4. Rendering results on the TUM dataset. The proposed keyframe-triggered single-step initialization produces sharper edges, fewer transparency artifacts, and more consistent colors than residual-driven densification.

4.4. Rendering Quality and Throughput

We report fidelity and throughput in Table 4 and 5. On Replica, our initializer averages **925 FPS**, exceeding MonoGS (769 FPS) while maintaining competitive PSNR [3], SSIM [19], and LPIPS [24] across *room0–2* and *office0–4*. On TUM, the system runs in real time (2.5–3.2 FPS) with PSNR/SSIM comparable to SplaTAM, PhotoSLAM, and GLORIE-SLAM, and low LPIPS. The throughput gain arises from the keyframe-triggered single-step den-

se initialization, which fixes Gaussian topology upfront and removes residual-driven densification, reducing per-frame cost. Qualitative results in Figure 4 show sharper edges, fewer transparency artifacts, and more consistent colors than residual-driven baselines.

Table 4. Rendering quality results on the Replica dataset across room0–2 and office0–4.

Method (FPS)	Metric	room0	room1	room2	office0	office1	office2	office3	office4	Avg.
NICE-SLAM [26] (6.54)	PSNR[dB]↑	22.12	22.47	24.52	29.07	30.34	19.66	22.23	24.94	24.42
	SSIM↑	0.689	0.757	0.814	0.874	0.868	0.797	0.801	0.856	0.809
	LPIPS↓	0.330	0.271	0.208	0.229	0.181	0.235	0.209	0.198	0.233
Vox-Fusion [21] (2.17)	PSNR[dB]↑	22.39	22.36	23.92	27.79	29.83	20.33	23.47	25.21	24.41
	SSIM↑	0.683	0.751	0.798	0.857	0.876	0.794	0.803	0.847	0.801
	LPIPS↓	0.303	0.269	0.234	0.241	0.184	0.243	0.213	0.199	0.236
Point-SLAM [12] (1.33)	PSNR[dB]↑	32.40	34.08	35.50	38.26	39.16	33.98	33.48	33.49	35.17
	SSIM↑	0.974	0.977	0.979	0.982	0.986	0.962	0.960	0.979	0.975
	LPIPS↓	0.113	0.116	0.111	0.100	0.118	0.156	0.132	0.142	0.124
Co-SLAM [18]	PSNR[dB]↑	28.88	28.51	29.37	35.44	34.63	26.56	28.79	32.16	28.42
	SSIM↑	0.892	0.843	0.851	0.854	0.826	0.814	0.866	0.856	0.837
	LPIPS↓	0.213	0.205	0.215	0.177	0.161	0.172	0.163	0.176	0.185
SplaTAM [7]	PSNR[dB]↑	32.49	33.72	34.65	38.29	39.04	31.91	30.05	31.83	30.98
	SSIM↑	0.975	0.970	0.980	0.982	0.982	0.965	0.952	0.949	0.953
	LPIPS↓	0.072	0.096	0.078	0.086	0.093	0.100	0.110	0.150	0.179
Gauss-SLAM [20]	PSNR[dB]↑	29.57	31.61	33.46	38.39	39.62	32.91	33.62	34.26	30.90
	SSIM↑	0.944	0.952	0.973	0.985	0.991	0.974	0.982	0.979	0.972
	LPIPS↓	0.197	0.184	0.148	0.099	0.097	0.158	0.123	0.138	0.229
MonoGS [11] (769)	PSNR[dB]↑	34.83	36.43	37.49	39.95	42.09	36.24	36.70	36.07	37.50
	SSIM↑	0.954	0.959	0.965	0.971	0.974	0.964	0.963	0.957	0.960
	LPIPS↓	0.068	0.076	0.075	0.072	0.055	0.078	0.065	0.099	0.070
Ours (RGB) (925)	PSNR[dB]↑	35.95	33.55	32.45	34.45	35.45	34.87	34.02	35.85	34.57
	SSIM↑	0.852	0.945	0.985	0.952	0.925	0.952	0.855	0.961	0.928
	LPIPS↓	0.085	0.092	0.112	0.088	0.096	0.078	0.101	0.096	0.093

Table 5. Rendering quality results on the TUM RGB-D dataset.

Method (FPS)	Metric	fr1/desk	fr2/xyz	fr3/office	Avg.
Point-SLAM [12]	PSNR[dB]↑	13.79	17.62	18.29	16.57
	SSIM↑	0.625	0.710	0.749	0.695
	LPIPS↓	0.545	0.584	0.452	0.527
Photo-SLAM [4]	PSNR[dB]↑	20.97	21.07	19.59	20.54
	SSIM↑	0.740	0.730	0.690	0.720
	LPIPS↓	0.230	0.170	0.240	0.213
MonoGS [11]	PSNR[dB]↑	19.67	16.17	20.63	18.82
	SSIM↑	0.730	0.720	0.770	0.740
	LPIPS↓	0.330	0.310	0.340	0.327
GLORIE-SLAM [23]	PSNR[dB]↑	20.26	25.62	21.21	22.36
	SSIM↑	0.790	0.720	0.720	0.743
	LPIPS↓	0.310	0.090	0.320	0.240
SplaTAM [7]	PSNR[dB]↑	21.49	25.06	21.17	22.57
	SSIM↑	0.839	0.950	0.861	0.883
	LPIPS↓	0.255	0.099	0.221	0.192
RK-SLAM [10]	PSNR[dB]↑	22.31	22.47	20.67	21.82
	SSIM↑	0.741	0.729	0.710	0.727
	LPIPS↓	0.254	0.220	0.251	0.242
Ours (RGB)	PSNR[dB]↑	23.11	24.85	23.59	23.85
	SSIM↑	0.853	0.896	0.801	0.850
	LPIPS↓	0.232	0.198	0.219	0.216

4.5. Reconstruction Fidelity

Geometric fidelity is evaluated using accuracy, completeness, and completeness ratio in Table 6, computed on aligned point clouds under standard thresholds. Our method attains **1.537 cm** accuracy and **1.477 cm** completeness with a **97.843%** completeness ratio. Relative to SNI-SLAM, accuracy improves by 20.9% and completeness by 13.2%, together with a 1.22-point gain in completeness ratio. The margins over ESLAM and Vox-Fusion are larger, including a 42% reduction in completeness error against Vox-Fusion. The improvements are consistent across scenes with thin structures and clutter, where coverage gaps and over-regularization commonly inflate geometric error. Qualitative inspection shows reduced truncation at object boundaries, cleaner reconstruction of high-frequency edges, and better recovery of small appendages. We attribute these outcomes to anisotropic Gaussian primitives with visibility-aware α -compositing, which sharpen depth gradients and limit color bleeding, and to a bounded, keyframe-related optimization that preserves spatial coverage without topology changes. By keeping the Gaussian set fixed after dense seeding, the optimization remains stationary and avoids late-map artifacts, which stabilizes surface inference and suppresses oversmoothing during refinement.

Table 6. Reconstruction results on the Replica dataset. Lower is better for Acc./Comp., higher for Comp.Ratio.

Methods	Reconstruction		
	Acc. [cm] ↓	Comp. [cm] ↓	Comp.Ratio (%) ↑
iMAP [16]	3.624	4.934	80.515
NICE-SLAM [26]	2.373	2.645	91.137
Vox-Fusion [21]	1.882	2.563	90.936
Co-SLAM [18]	2.104	2.082	93.435
ESLAM [6]	2.082	1.754	96.427
SNI-SLAM [25]	1.942	1.702	96.624
Ours	1.537	1.477	97.843

4.6. Ablation Study

Effect of Dense Initialization. Consistent rendering gains on TUM RGB, with higher PSNR/SSIM and lower LPIPS/RMSE across all sequences. On *fr1.desk*, *fr2.xyz*, and *fr3.office*, PSNR improves to 23.11, 24.85, and 23.59 with SSIM gains and LPIPS/RMSE drops (Table 7). Distributed Gaussian seeds, whose multi-view triangulation stabilizes mapping under larger motion and maintains coverage in low-parallax segments. This yields faster convergence and fewer artifacts on thin structures and cluttered regions. Without dense initialization, residual driven densification converges slowly, exhibits early spatial inconsistency, and tends to over-smooth before adequate coverage is established.

Table 7. Impact of DFM on the TUM RGB-D dataset.

	Method	PSNR ↑	SSIM ↑	LPIPS ↓	RMSE ↓
fr1_desk	w/o DFM	19.67	0.73	0.33	1.5
	Ours	23.11	0.853	0.232	1.02
fr2_xyz	w/o DFM	16.17	0.72	0.31	1.44
	Ours	24.85	0.896	0.198	0.98
fr3_office	w/o DFM	20.63	0.77	0.34	1.49
	Ours	23.59	0.801	0.219	1.05

Effect of Gaussian Count per Keyframe on Tracking.

We vary the number of newly triangulated 3D points per keyframe, with each verified point instantiated as a Gaussian primitive, so the abscissa in Figure 5 corresponds to the count of Gaussians. Increasing the budget from 200 to 1000 points reduces the tracking error sharply, reaching about **0.7cm** at 1000. Beyond this regime the curve plateaus and improvements are marginal, approaching roughly 0.6 cm at 2000. We therefore adopt 1000 points per keyframe as the default trade-off between accuracy, memory, and runtime.

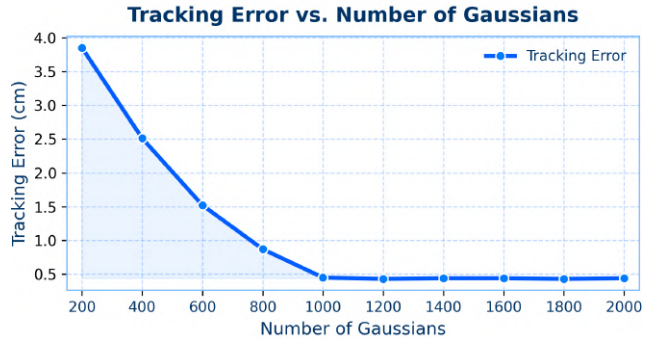


Figure 5. Tracking error versus Gaussian count. Error decreases rapidly with denser seeding and plateaus near 1000 Gaussians, indicating diminishing returns beyond this density.

5. Conclusion

This paper introduced CoGS-SLAM, a Gaussian-splatting SLAM approach that removes residual-based densification in favor of a keyframe-driven one-shot initialization strategy. Through the combination of dense correspondence estimation and multi-view triangulation, the method constructs reliable Gaussian priors that support differentiable optimization with analytic SE(3) Jacobians. Results on Replica and TUM RGB-D show that the proposed system improves computational efficiency while maintaining strong localization and rendering performance. Since the overall formulation remains compatible with established SLAM pipelines, CoGS-SLAM provides a practical and scalable direction for differentiable 3D mapping.

References

- [1] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superpoint: Self-supervised interest point detection and description. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018. 2
- [2] Johan Edstedt, Ioannis Athanasiadis, Mårten Wadenbäck, and Michael Felsberg. DKM: Dense kernelized feature matching for geometry estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
- [3] Alain Hore and Djemel Ziou. Image quality metrics: PSNR vs. SSIM. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2010. 5, 7
- [4] Huajian Huang, Longwei Li, Hui Cheng, and Sai-Kit Yeung. Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo, and RGB-D cameras. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2, 5, 7
- [5] Jiahui Huang, Shi-Sheng Huang, Haoxuan Song, and Shi-Min Hu. DI-Fusion: Online implicit 3D reconstruction with deep priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 6
- [6] Mohammad Mahdi Johari, Camilla Carta, and François Fleuret. ESLAM: Efficient dense SLAM system based on hybrid representation of signed distance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 5, 6, 8
- [7] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. Splatam: Splat, track & map 3D Gaussians for dense RGB-D SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2, 5, 7
- [8] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 2023. 2
- [9] Philipp Lindenberger, Paul-Edouard Sarlin, and Marc Pollefeys. Lightglue: Local feature matching at light speed. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 2
- [10] Xiasheng Ma, Ci Song, Yimin Ji, and Shanlin Zhong. Related keyframe optimization Gaussian-simultaneous localization and mapping: A 3D Gaussian Splatting-based simultaneous localization and mapping with related keyframe optimization. *Applied Sciences*, 2025. 5, 7
- [11] Hidenobu Matsuki, Riku Murai, Paul Kelly, and Andrew J. Davison. Gaussian Splatting SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2, 5, 6, 7
- [12] Erik Sandström, Yue Li, Luc Van Gool, and Martin R. Oswald. Point-SLAM: Dense neural point cloud-based SLAM. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 2, 5, 6, 7
- [13] Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superglue: Learning feature matching with graph neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 2
- [14] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 2
- [15] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 4
- [16] Edgar Sucar, Shikun Liu, Joseph Ortiz, and Andrew J. Davison. imap: Implicit mapping and positioning in real-time. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. 5, 6, 8
- [17] Jiaming Sun, Zehong Shen, Yuang Wang, Hujun Bao, and Xiaowei Zhou. LoFTR: Detector-free local feature matching with transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2
- [18] Hengyi Wang, Jingwen Wang, and Lourdes Agapito. Co-SLAM: Joint coordinate and sparse parametric encodings for neural real-time SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 5, 6, 7, 8
- [19] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. 5, 7
- [20] Chao Yan, Zirui Wang, Zhiqiang Li, Wei Gao, Hao Wang, Guofeng Zhang, Hujun Bao, and Xiaowei Zhou. GS-SLAM: Dense visual SLAM with 3D Gaussian Splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2, 5, 7
- [21] Xingrui Yang, Hai Li, Hongjia Zhai, Yuhang Ming, Yuqian Liu, and Guofeng Zhang. Vox-fusion: Dense tracking and mapping with voxel-based neural implicit representation. In *Proceedings of the IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 499–507, 2022. 5, 6, 7, 8
- [22] Yao Yao, Zixin Luo, Shiwei Li, Tian Fang, and Long Quan. MVSNet: Depth inference for unstructured multi-view stereo. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018. 2
- [23] Ganlin Zhang, Erik Sandström, Youmin Zhang, Manthan Patel, Luc Van Gool, and Martin R. Oswald. GLORIE-SLAM: Globally optimized RGB-only implicit encoding point cloud SLAM. *arXiv preprint arXiv:2403.19549*, 2024. 2, 5, 7
- [24] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of

deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. [5](#), [7](#)

- [25] Siting Zhu, Guangming Wang, Hermann Blum, Jiuming Liu, Liang Song, Marc Pollefeys, and Hesheng Wang. SNI-SLAM: Semantic neural implicit SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21167–21177, 2024. [5](#), [8](#)
- [26] Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R. Oswald, and Marc Pollefeys. NICE-SLAM: Neural implicit scalable encoding for SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. [5](#), [6](#), [7](#), [8](#)

CoGS-SLAM: Correspondence-Guided Gaussian Splatting SLAM with One-Shot Dense Initialization

Supplementary Material

6. Reproducibility and Code Availability

All components of the system are implemented in PyTorch with custom CUDA kernels for rasterization and analytic Jacobians. The repository provides the full SLAM pipeline, configuration files, scripts for evaluation on TUM RGB-D and Replica, and instructions for reproducing all quantitative tables and qualitative renderings in the main paper. The training and optimization settings match those described in Sections 3 and 4 of the main paper.

7. Discussion

7.1. Limitations

CoGS-SLAM still exhibits several practical limitations despite the gains over residual-driven densification. The current evaluation focuses on indoor static scenes in Replica and TUM RGB-D, so the robustness of the one-shot Gaussian initialization in highly dynamic scenes, or under severe rolling shutter remains unclear. The dense correspondences are derived from a single pretrained DINOv3 backbone together with a confidence aware inlier classifier, which can degrade under drastic appearance changes, strong motion blur, or camera viewpoints far outside the training distribution, and in these regimes the triangulated seeds may become biased or incomplete. The fixed topology after each keyframe initialization improves stability but can leave persistent coverage gaps when large textureless surfaces, fine specular structures, or objects with weak visual support never accumulate enough consistent matches, which constrains the reconstruction quality. The current implementation also assumes a calibrated pinhole camera and synchronized RGB-D streams, without exploiting lidar cues or inertial measurements that are available on many robotic platforms. Finally, the system still requires a GPU with moderate memory to sustain high frame rates, and the practical impact of memory budgets, long-term map growth, and large-scale loop closure has not yet been characterized on resource constrained devices.

7.2. Societal Impact

CoGS-SLAM advances camera based dense SLAM with a training-free one-shot Gaussian initialization that stabilizes optimization and improves throughput, which can benefit robotics, extended reality, and digital twin systems through more reliable mapping and safer physical interaction. At the same time, dense reconstruction of indoor environments raises privacy risks whenever RGB or RGB-D streams are

captured and stored without informed consent, since metrically consistent maps can reveal layouts, personal belongings, and usage patterns. The method is training-free and easily integrated into existing SLAM stacks, which lowers the barrier for large-scale deployment and makes responsible use dependent on appropriate safeguards such as transparent data handling, limited retention of raw sensor data, access control to stored maps, and a preference for local processing. Our experiments rely on public benchmarks without personally identifiable information and the code release is intended for reproducible research, although future work should pair technical advances in robust mapping with privacy aware data design and interdisciplinary guidelines for ethical deployment.

7.3. Training Details

Optimization of Gaussian maps. The optimization settings used by our Gaussian SLAM system are summarized in Table 8. For both initialization and online mapping we use the Adam optimizer with a shared parameterization for geometry and appearance. The base learning rate is set to 2.0×10^{-3} for color and opacity parameters and 1.0×10^{-3} for positions and rotations, with $(\beta_1, \beta_2) = (0.9, 0.999)$. A cosine decay schedule is applied within each optimization window so that early iterations focus on rapid geometry refinement while later iterations stabilize the map. Initialization uses a window of five frames and runs for 1050 iterations before the system starts live tracking. During online operation every incoming frame is refined for 30 tracking iterations and each accepted keyframe-triggers 60 mapping iterations over the same local window. The loss combines an L_1 term and an SSIM term weighted by $\lambda_{\text{dssim}} = 0.2$. Dynamic density control uses the same thresholds across all experiments and adopts the opacity culling and densification settings listed in Table 8. Mixed precision training and gradient norm clipping are enabled to keep the per frame compute budget and memory stable.

Dense initialization baseline. As an additional baseline we adopt a dense correspondence based initialization strategy that shares the same differentiable Gaussian representation as our system. The training configuration is summarized in Table 9. The model is optimized for 30000 iterations with Adam and separate learning rates for spatial and appearance parameters. Position updates follow a decayed schedule from 1.6×10^{-4} to 1.6×10^{-6} , while feature, opacity, scaling, and rotation parameters use constant rates that match the public configuration. The reconstruction loss

Table 8. Optimization configuration for the Gaussian SLAM.

Config	Value
Optimizer	Adam
Adam betas	$\beta_1 = 0.9, \beta_2 = 0.999$
Base LR (color, opacity)	2.0×10^{-3}
Base LR (position, rotation)	1.0×10^{-3}
Learning rate schedule	Cosine decay
Loss	$L_1 + \lambda_{\text{dssim}} L_{\text{SSIM}}$
SSIM weight λ_{dssim}	0.2
Initialization iterations	1050
Tracking iterations per frame	30
Mapping iterations per keyframe	60
Local window size	5 frames
Densification interval	100 iterations
Opacity reset interval	3000 iterations
Opacity culling threshold	0.05
Gradient clipping max norm	1.0
Precision	Mixed FP16 / FP32

combines an ℓ_1 term with a structural similarity component with $\lambda_{\text{dssim}} = 0.2$. Densification is enabled from iteration 500 to 15000 with an interval of 100 iterations and a gradient based threshold of 2.0×10^{-4} . All experiments use degree three spherical harmonics, batch size 64, and a fixed background color without random perturbations on a CUDA device.

8. Implementation Details

Datasets. We evaluate on the TUM RGB-D and Replica benchmarks, consistent with Section 4 of the main paper. TUM RGB-D is used in both monocular and RGB-D configurations, following standard practice in visual SLAM. Replica is employed for photometric map evaluation and trajectory accuracy on the eight standard indoor scenes *room0* to *room2* and *office0* to *office4*. These splits match those used in the main tables so that rendering metrics, throughput, and localization error remain directly comparable across methods.

Evaluation Metrics. Camera tracking accuracy is measured using the root mean square error of the Absolute Trajectory Error (ATE) computed on keyframes. Photometric map quality is evaluated with three rendering metrics: PSNR, SSIM, and LPIPS. Geometric reconstruction quality is assessed with accuracy (Acc. [cm]), completeness (Comp. [cm]), and completeness ratio (Comp.Ratio [%]). Accuracy is defined as the mean nearest-neighbour distance from reconstructed points to the ground-truth surface. Completeness is the mean nearest-neighbour distance from ground-truth surface samples to the reconstruction. The completeness ratio is the proportion of ground-truth sam-

Table 9. Training configuration of the dense initialization baseline.

Config	Value
Total iterations	30000
Optimizer	Adam
Spherical harmonics degree	3
Batch size	64
Position LR (init $\rightarrow$ final)	$1.6 \times 10^{-4} \rightarrow 1.6 \times 10^{-6}$
Feature learning rate	2.5×10^{-3}
Opacity learning rate	2.5×10^{-2}
Scaling learning rate	5.0×10^{-3}
Rotation learning rate	1.0×10^{-3}
SSIM weight λ_{dssim}	0.2
Dense correspondence ratio	0.01
Densification interval	100 iterations
Densification range	iter. 500 to 15000
Densification gradient threshold	2.0×10^{-4}
Opacity reset iteration	30000
Exposure LR (init $\rightarrow$ final)	$1.0 \times 10^{-2} \rightarrow 1.0 \times 10^{-4}$
Random background	disabled
Logging train camera index	50
Logging test camera index	10
Dataset resolution	original
White background	false
Data device	CUDA

ples within a distance threshold τ from the reconstruction. Unless stated otherwise, we uniformly sample 50K points on each surface, set $\tau = 5$ cm, and average per-scene scores over the benchmark sequences. For photometric rendering metrics, we evaluate every fifth frame and exclude keyframes to avoid bias toward training views. Reconstruction metrics are computed with the same surface-sampling protocol. Each experiment is repeated three times on identical hardware and stopping criteria, and all tables report the mean scores. In every table, the best result is typeset in bold and the second best is underlined.

8.1. Compare Model Settings

We compare CoGS-SLAM with a set of representative dense SLAM pipelines that cover implicit volumes, voxel grids, point clouds, and Gaussian splats under the same scene types and benchmarks. NICE-SLAM, Co-SLAM, Vox-Fusion, DI-Fusion, and SNI-SLAM operate on RGB-D input and maintain volumetric implicit or voxel based representations that are optimized per scene. iMAP and Point-SLAM map monocular or RGB-D streams to neural fields or point clouds with scene specific training and joint pose refinement. SplatTAM, Gauss-SLAM, MonoGS, and RK-SLAM adopt 3D Gaussian splatting and couple a Gaussian renderer with pose and appearance optimization, while Photo-SLAM and GLORIE-SLAM combine differentiable

rendering with explicit point or mesh structures.

8.2. Computing Resource Configuration

All experiments are run on a workstation equipped with two NVIDIA L40 GPUs and an Intel Xeon Platinum 8362 CPU at 2.80 GHz. Time-critical components, including 3D Gaussian rasterization and gradient computation, are implemented in CUDA, while the remaining SLAM pipeline is implemented in PyTorch. Mixed precision is enabled for rendering and backpropagation whenever this improves throughput without degrading stability. The tracking loop operates in real-time, and mapping is executed asynchronously within a bounded local window so that latency remains stable as the map grows.

9. Methodology Details

9.1. Gaussian Splatting Representation

Each Gaussian G_i is represented by a compact tuple containing the world space mean $\mu_i^W \in \mathbb{R}^3$, an opacity parameter $\alpha_i \in [0, 1]$, a set of view dependent color coefficients, and a covariance parameterization. In the underlying model the covariance appears as a full matrix Σ_i^W , while in the implementation it is encoded as a rotation matrix $R_i \in \text{SO}(3)$ and three axis aligned scales $s_i \in \mathbb{R}_+^3$ such that

$$\Sigma_i^W = R_i \text{diag}(s_i^2) R_i^\top. \quad (15)$$

The rotation is stored as a unit quaternion and the scales are optimized in logarithmic space, which guarantees positive definiteness under gradient updates.

Colors are represented by second order spherical harmonics in camera space. For a viewing direction $\mathbf{v} \in \mathbb{S}^2$, the color of G_i is

$$C_i(\mathbf{v}) = \sum_{\ell=0}^2 \sum_{m=-\ell}^{\ell} \mathbf{c}_{i,\ell m} Y_\ell^m(\mathbf{v}), \quad (16)$$

where $\mathbf{c}_{i,\ell m} \in \mathbb{R}^3$ are learned RGB coefficients and Y_ℓ^m are real spherical harmonics.

Projection to the image plane uses the calibrated camera intrinsics together with the Gaussian projection model, and screen space compositing applies standard alpha compositing. All attributes are packed into contiguous GPU buffers and updated in place. This layout lets the renderer handle several million Gaussians without fragmentation and keeps memory access patterns coherent during both forward and backward passes.

9.2. Tracking and Camera Pose Optimization

Given the current map $\mathcal{G}$ and a new RGB or RGB-D frame, pose tracking alternates between rendering and gradient based refinement of the camera pose T_{WC} . We render

a synthesized image $\hat{I}(x; \mathcal{G}, T_{WC})$ at the native resolution and apply an affine brightness model with gain g_I and bias b_I for each frame,

$$\tilde{I}_I(x; \mathcal{G}, T_{WC}) = g_I \hat{I}(x; \mathcal{G}, T_{WC}) + b_I. \quad (17)$$

The parameters (g_I, b_I) are estimated by a small least squares problem

$$(g_I, b_I) = \arg \min_{g, b} \sum_{x \in \Omega_I} \left(I(x) - g \hat{I}(x; \mathcal{G}, T_{WC}) - b \right)^2, \quad (18)$$

and the resulting solution is substituted into the photometric objective so that pose updates remain invariant to slow exposure drift.

The tracking loss can be written in the form

$$\mathcal{L}_{\text{track}}(T_{WC}) = \sum_{x \in \Omega_I} w_I(x) \|I(x) - \tilde{I}_I(x; \mathcal{G}, T_{WC})\|_1, \quad (19)$$

where $w_I(x)$ denotes a per pixel weight. In practice this weight is factored into opacity and gradient terms,

$$w_I(x) = w_\alpha(x) w_\nabla(x), \quad (20)$$

with

$$w_\alpha(x) = \text{clip} \left(\frac{\hat{\alpha}(x)}{\tau_\alpha}, 0, 1 \right), \quad (21)$$

$$w_\nabla(x) = \text{clip} \left(\frac{\|\nabla I(x)\|_2}{\tau_\nabla}, 0, 1 \right), \quad (22)$$

where $\hat{\alpha}(x)$ is the accumulated opacity at pixel x , τ_α and τ_∇ are fixed thresholds, and $\text{clip}(z, a, b) = \min(\max(z, a), b)$. Pixels with low opacity or weak gradients therefore have reduced influence in the optimization.

The pose is updated in the minimal twist coordinates $\xi \in \mathbb{R}^6$ using a standard left multiplicative update rule on $\text{SE}(3)$. The Jacobians of the camera projection and the image formation model are implemented analytically and are reused across all Gaussians that share the same pose, which reduces both computation and memory traffic. The derivative of the projected covariance with respect to pose is evaluated by an explicit chain rule involving the projection Jacobian and the local rotation. This avoids generic automatic differentiation through the entire renderer.

In practice, between thirty and sixty gradient steps per frame are performed using the Adam optimizer with a cosine learning rate schedule centered around 5×10^{-3} . This configuration yields stable tracking even when large parts of the image are textureless or underexposed.

9.3. Keyframe Scheduling by Co-Visibility

Keyframe scheduling uses a co-visibility based policy. For each incoming frame we maintain a binary visibility mask

that records, for every pixel, whether its accumulated screen space opacity exceeds a small threshold. Let V_a and V_b denote the sets of visible pixels in images I_a and I_b . The co-visibility score is defined as

$$\text{IoU}(I_a, I_b) = \frac{|V_a \cap V_b|}{|V_a \cup V_b|}. \quad (23)$$

The intersection and union are counted entirely on the GPU.

A frame I_k is promoted to a keyframe when the co-visibility $\text{IoU}(I_k, I_{k^*})$ with the last keyframe I_{k^*} falls below a user defined threshold τ and when the relative translation and rotation exceed small geometric bounds. These additional bounds avoid accepting nearly redundant viewpoints that would increase memory and computation without improving triangulation baselines.

Accepted keyframes are stored in a bounded buffer $\mathcal{B}$ together with their poses. Between eight and twelve recent keyframes are kept, which is sufficient to form well-conditioned local triangulation baselines while keeping all multi view operations inexpensive. The same buffer also provides the neighbour set used in the mapping stage.

9.4. Dense Feature Matching and Triangulation

Dense matching and multi view initialization rely on DINOv3 features computed at a fixed feature resolution obtained by downsampling the RGB images. The descriptors are ℓ_2 normalized per pixel. For each reference keyframe I_r a neighbour set $\mathcal{N}_r \subset \mathcal{B}$ is selected based on pose proximity and parallax, using the current camera estimates.

Let $f_r(p)$ and $f_n(q)$ denote DINOv3 descriptors at pixels p and q in images I_r and I_n . For each neighbour $n \in \mathcal{N}_r$ and displacement $\mathbf{u}_{r \rightarrow n}(p)$, the raw descriptor similarity is

$$s_{r \rightarrow n}(p) = \langle f_r(p), f_n(p + \mathbf{u}_{r \rightarrow n}(p)) \rangle. \quad (24)$$

This similarity is combined with forward backward consistency and epipolar agreement into a scalar score

$$\tilde{\kappa}_{r \rightarrow n}(p) = w_{\text{sim}} s_{r \rightarrow n}(p) + w_{\text{fb}} \rho_{\text{fb}}(p) + w_{\text{epi}} \rho_{\text{epi}}(p), \quad (25)$$

where ρ_{fb} and ρ_{epi} quantify consistency of the forward backward displacement and the epipolar distance, and w_{sim} , w_{fb} , and w_{epi} are fixed weights. The confidence $\kappa_{r \rightarrow n}(p)$ is obtained by a piecewise linear mapping to $[0, 1]$,

$$\kappa_{r \rightarrow n}(p) = \text{clip} \left(\frac{\tilde{\kappa}_{r \rightarrow n}(p) - \gamma_0}{\gamma_1 - \gamma_0}, 0, 1 \right), \quad (26)$$

with fixed thresholds γ_0 and γ_1 . This design keeps the inlier classifier training-free and dataset agnostic.

Before triangulation, correspondences are thinned in image space with a blue noise pattern in order to avoid redundant seeds in locally homogeneous regions. The aggregated confidence $\bar{\kappa}(p)$ is cached for each surviving refer-

ence pixel and encodes agreement across multiple neighbours. Linear triangulation uses a homogeneous linear system solved with a double precision singular value decomposition. Candidates with very small parallax or a reprojection error above a conservative threshold are discarded, and among hypotheses obtained from different neighbours the one with the smallest reprojection error and a sufficiently large baseline angle is kept.

9.5. Gaussian Initialization and Joint Mapping

Each valid triangulated point spawns a Gaussian G_i whose world mean is set to the reconstructed point $\mu_i^W = \hat{X}(p)$. A local orthonormal frame $U_i = [\mathbf{t}_1, \mathbf{t}_2, \mathbf{v}]$ is constructed by fitting a plane to triangulated neighbours inside a fixed radius around $\hat{X}(p)$, where $\mathbf{v}$ is the normal and $\mathbf{t}_1, \mathbf{t}_2$ span the tangent plane in the neighbourhood. The covariance is initialized as an anisotropic ellipsoid aligned with this frame,

$$\Sigma_i^W = U_i \text{diag}(s_{\perp}^2, s_{\perp}^2, s_{\parallel}^2) U_i^{\top}, \quad (27)$$

where the tangential scale $s_{\perp}$ is obtained by back projecting a one pixel footprint through the projection Jacobian at the reference view and the axial scale $s_{\parallel}$ is a calibrated function of the baseline angle and triangulation residual, which increases depth uncertainty in poorly conditioned configurations.

The initial color c_i is the median of bilinear RGB samples across all supporting views after applying the exposure parameters estimated during tracking. The initial opacity α_i is obtained as a monotone mapping of $\bar{\kappa}(p)$,

$$\alpha_i = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \bar{\kappa}(p), \quad (28)$$

with fixed bounds $\alpha_{\min}$ and $\alpha_{\max}$ in $(0, 1)$. Unreliable seeds with low aggregated confidence therefore enter the map as visually weak Gaussians and are easy to prune. Before insertion, Poisson disk subsampling in world space is applied to promote uniform coverage and to avoid excessive density in locally flat regions.

After insertion, the mapping module maintains an observation count m_i , a cumulative visibility score v_i , and an exponential moving average of the world position $\bar{\mu}_i^W$ for each Gaussian. The moving average is updated after each mapping step according to

$$\bar{\mu}_i^{W(t+1)} = (1 - \eta) \bar{\mu}_i^{W(t)} + \eta \mu_i^{W(t+1)}, \quad (29)$$

with a fixed smoothing factor $\eta \in (0, 1)$.

The mapping loss is evaluated over a sliding window $\mathcal{W}$ around the latest keyframe with per frame brightness parameters g_I and b_I and per pixel weights $w_I(x)$ as in Eq. (20). A regularizer discourages highly elongated covariances, avoids degenerate transmittance, and anchors early updates to $\bar{\mu}_i^W$. The coefficients of this regularizer are kept fixed across all sequences and are not adapted per scene,

which keeps the behaviour of the optimizer comparable on TUM and Replica.

Pose only updates and full map updates are alternated, and robust per pixel weights are used so that outliers in the photometric residual have limited influence. A lightweight merging operation averages color and covariance for pairs of Gaussians that have almost identical screen space footprints and similar appearance. Pruning removes Gaussians that violate bounds on m_i , v_i , $\text{tr}(\Sigma_i^W)$, or α_i . These maintenance steps keep the representation compact and prevent numerical instabilities caused by extremely large or extremely transparent splats.

9.6. System Schedule and Computational Profile

The runtime schedule separates tracking and mapping. Each incoming frame is tracked for K_t gradient steps using the photometric loss described above. This stage updates only the current pose and does not modify the map. If the co-visibility test does not accept the frame as a keyframe, the system immediately proceeds to the next image.

Let ρ denote the empirical fraction of frames that are selected as keyframes. The average number of gradient based optimization steps per frame is then approximately

$$N_{\text{steps}}^{\text{frame}} \approx K_t + \rho K_m, \quad (30)$$

where K_m is the number of joint refinement iterations executed after each keyframe.

When a keyframe is accepted, a set of neighbours from $\mathcal{B}$ is selected and dense matching, confidence aggregation, triangulation, and Gaussian initialization are executed in a single GPU pass. The resulting Gaussians are inserted into $\mathcal{G}$ and immediately participate in mapping. The system then runs K_m iterations of joint refinement over the window $\mathcal{W}$ using the mapping loss and regularizer defined above, followed by a maintenance pass that performs merging and pruning.

This schedule replaces iterative residual-driven densification with a one-shot dense seed and thereby reduces the early drift of newly added parameters. The number of mapping iterations required to reach a given photometric fidelity decreases, which improves wall clock throughput while keeping the renderer and optimization objectives identical to those used in the main method description.

10. Additional Experiments

10.1. Additional Rendering on Replica

On the Replica dataset, extended qualitative comparisons in Figure 6 show that our keyframe-triggered single-step initialization yields sharper object boundaries and more stable shading than the residual-driven densification baseline across living room and office scenes. The reconstructed

views exhibit fewer transparency artifacts around thin structures such as chair legs and table edges, and color transitions remain consistent across viewpoints, which confirms that the proposed initialization creates a well-conditioned Gaussian map for subsequent optimization. Challenging regions for Gaussian splatting, including large textureless walls and slanted ceilings, also show reduced blotchy artifacts because the one-shot dense seed avoids early gaps in coverage. These qualitative trends agree with the quantitative gains in reconstruction metrics reported in the main paper and indicate that the initializer improves both convergence speed and the final visual fidelity of the radiance field.

10.2. Camera Tracking on Replica Offices

To assess tracking robustness, we visualize top view camera trajectories for two Replica office scenes in Figure 7 and 8. The proposed system maintains tight alignment with ground truth over long paths that include turns, loops, and revisits, and the strong overlap between the predicted and reference trajectories indicates that the jointly optimized poses and Gaussians provide accurate geometric constraints for downstream mapping and loop closure. In the office0 sequence the path combines slow pans and rapid rotations around the desk area, yet our trajectory returns to previously visited regions without noticeable misalignment, while competitor methods accumulate drift near corners and walls. In the office2 sequence the camera passes through narrow corridors before entering a wider workspace, and the estimated path from our method preserves the global layout without evident shearing or scale distortion. These qualitative patterns are consistent with the Absolute Trajectory Error reported in the main paper and support the claim that dense Gaussian initialization yields a stable optimization landscape for pose refinement.

10.3. Cluttered Desk Reconstruction

In a cluttered desk sequence with strong self occlusion and fine scale objects such as cables, pencils, and plush toys, shown in Figure 9, we evaluate an additional reconstruction. The left image shows the input RGB view and the right image illustrates the corresponding Gaussian map rendered from a novel viewpoint. The reconstruction preserves thin structures and surface boundaries while avoiding the truncation and over smoothing artifacts observed in residual driven pipelines, which demonstrates that the proposed initialization and refinement strategy scales to scenes with complex object layouts and high frequency details. Fine elements such as tripod legs, monitor edges, and scattered stationery remain clearly separated from the background even when objects move partially in and out of view, so the map integrates evidence from multiple viewpoints without duplicated Gaussians.

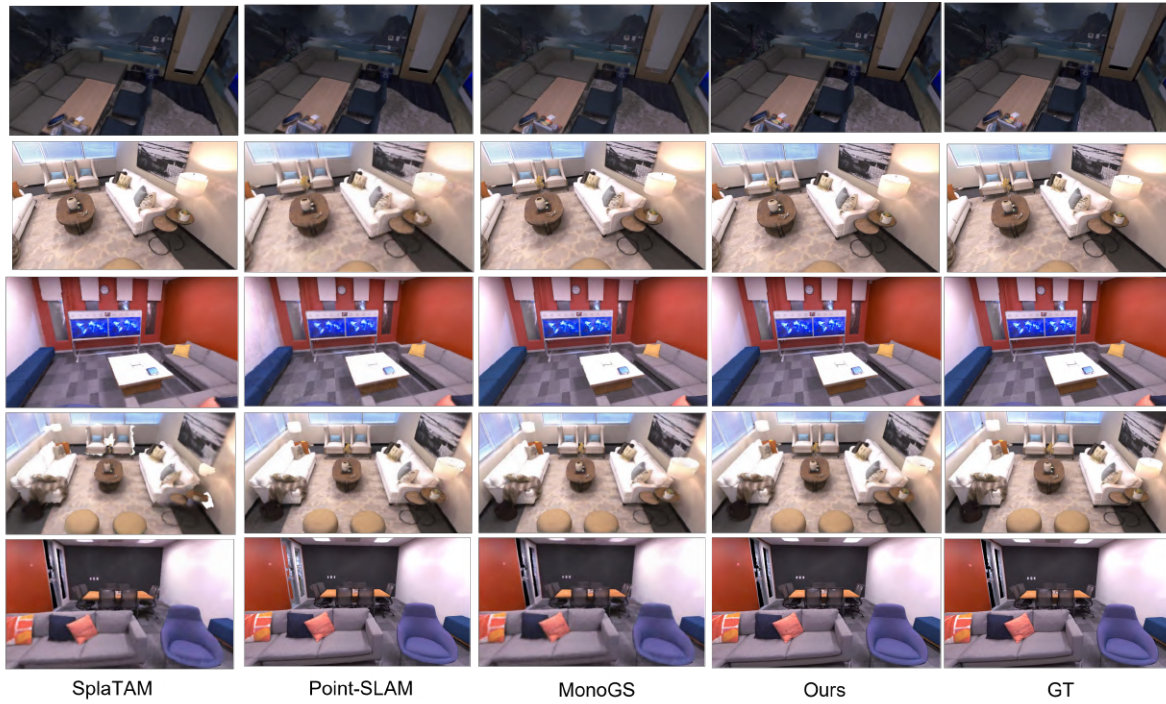


Figure 6. Rendering results on the Replica dataset. The proposed keyframe-triggered single-step initialization produces sharper edges, fewer transparency artifacts, and more consistent colors than residual-driven densification.

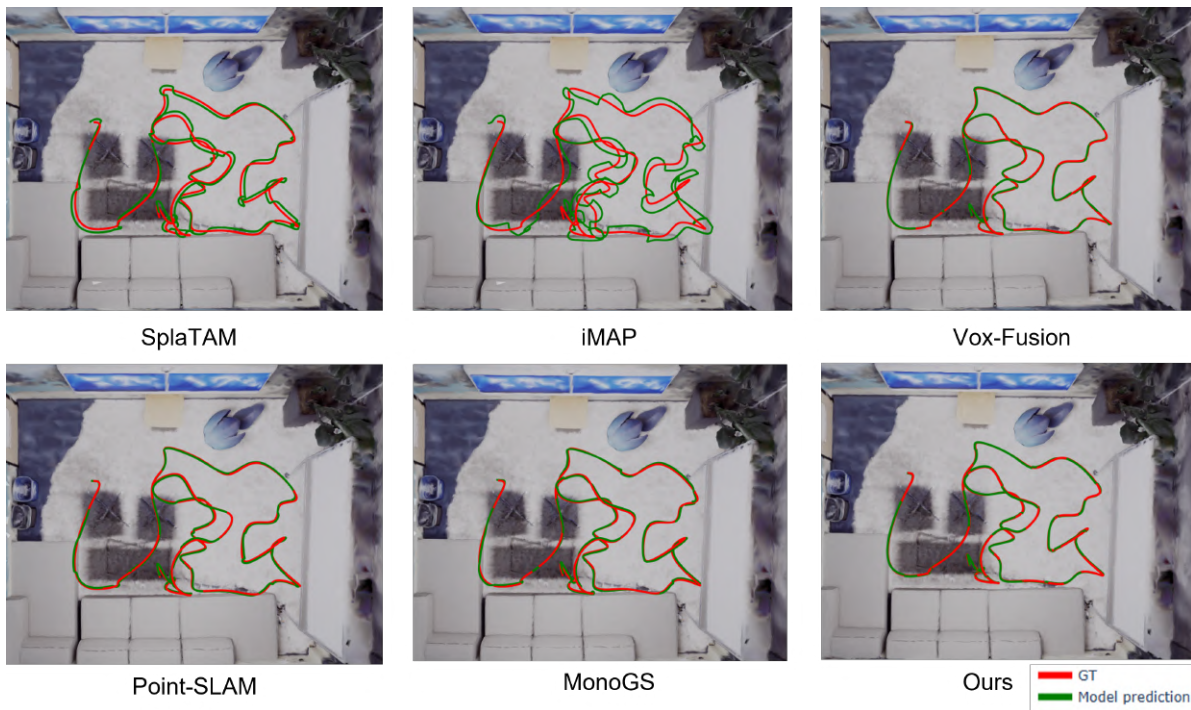


Figure 7. Tracking on Replica office0. Top-view trajectories, with ground truth in red and model predictions in green.

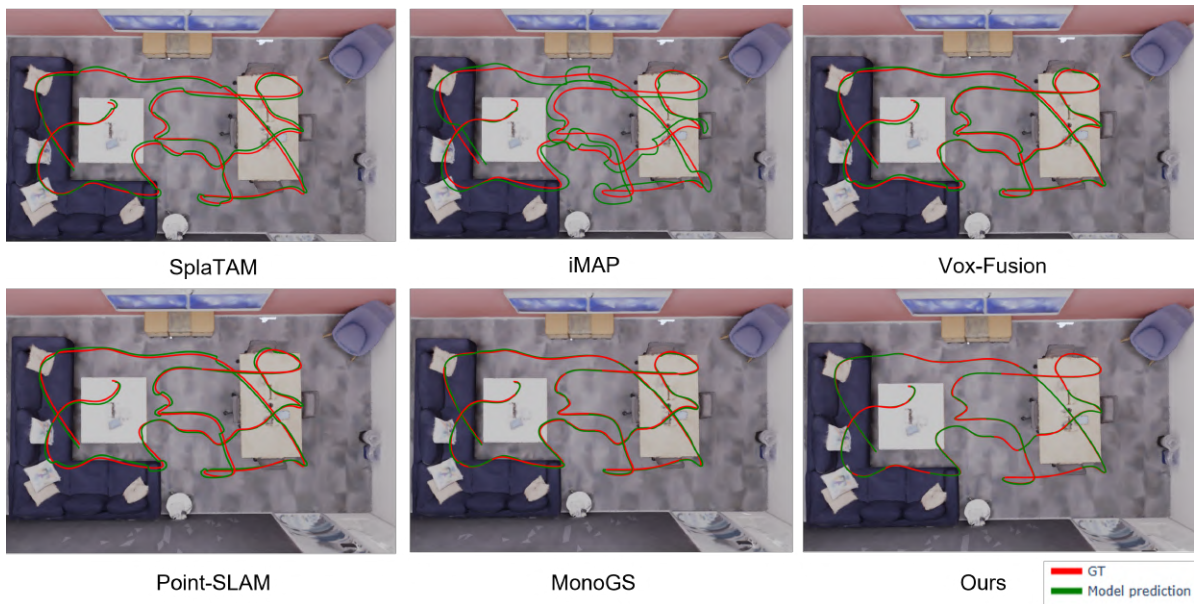


Figure 8. Tracking on Replica office2. Top-view trajectories, with ground truth in red and model predictions in green.



Figure 9. Qualitative reconstruction on a cluttered desk scene, where the left input RGB frame and the right Gaussian map view demonstrate dense coverage with preserved fine structures and reduced truncation artifacts.